

Sound Permission Verification for MCP Servers

Static Capability Inference vs. Declared Tool Descriptions

NURSULTAN SAGYNTAY, New York University Abu Dhabi,

SHAF KHALID, New York University Abu Dhabi,

MUHAMMAD SHAHZAIB HASSAN, New York University Abu Dhabi,

The Model Context Protocol (MCP) lets large language model agents invoke external tools advertised through natural-language descriptions. The hosting agent, the user, and any layered audit infrastructure rely on those descriptions to decide whether a tool is safe to call. We argue that this trust model is unsound by construction: nothing in the protocol forces the declared description and the implementing code to agree. A tool advertised as “reads a file” is free to launch a shell, open a database connection, or exfiltrate environment variables.

We present MCP-PERM CHECK, a static analyser that recovers the *actual* capability set P_{Actual} of every tool exported by a TypeScript MCP server and checks the inclusion $P_{\text{Actual}} \subseteq P_{\text{Declared}}$ over a 13-leaf hierarchical capability lattice. The analyser builds an explicit, materialised call graph from the TypeScript Compiler API, follows cross-module function calls through the type checker, and resolves I/O sinks against a curated catalog of 70 entries spanning the Node standard library, common npm clients (axios, undici, node-fetch, ws, mongodb, mongoose, redis, ioredis, nodemailer), and Prisma/SQL flow-sensitive shapes. Four shape adapters cover the dominant tool-registration idioms in the wild: `server.tool`, `server.registerTool`, `setRequestHandler(ListToolsRequestSchema, . . .)` with a switch dispatcher, and `fastmcp’s addTool(execute)`.

We evaluate on a 500-server subset of the public MCPCorpus dataset. The analyser successfully extracts capabilities for 84 servers (648 tools), of which 243 tools (37.5%) violate the inclusion. The most commonly undeclared leaves are `NETWORK_OUTBOUND` (139), `ENV` (47), `WRITE_DB` (46), and `WRITE_FS` (45). These violations are witnessed by exact source locations rather than embedding-similarity heuristics, so an auditor can confirm or dismiss each finding by reading a single line of code. Compared with prior work that flags suspicious mismatches through cosine similarity between description and code embeddings, our analysis produces concrete, machine-checkable witnesses and is sound under the stated coverage assumptions.

Additional Key Words and Phrases: Static analysis, Model Context Protocol, capability lattice, permission verification, call-graph construction, TypeScript

ACM Reference Format:

Nursultan Sagyntay, Shaf Khalid, and Muhammad Shahzaib Hassan. 2026. Sound Permission Verification for MCP Servers: Static Capability Inference vs. Declared Tool Descriptions. In *Proceedings of Static Program Analysis Course Project (OOPSLA 2026 Class Project)*. ACM, New York, NY, USA, 16 pages.

1 Introduction

1.1 Context

The Model Context Protocol (MCP) has become the de-facto integration boundary between large-language-model (LLM) agents and external tools [Anthropic 2024]. An MCP *server* is a small program — typically a single TypeScript or Python file — that registers a set of *tools*. Each tool is a record carrying a string name, a free-form description, an input schema, and a handler. When an agent decides to call a tool, it consults the description to predict what the tool will do and to obtain user consent. There is no standardised effect annotation, no sandbox, and no enforcement layer between the description and the handler.

Authors’ Contact Information: Nursultan Sagyntay, New York University Abu Dhabi, , , ; Shaf Khalid, New York University Abu Dhabi, , , ; Muhammad Shahzaib Hassan, New York University Abu Dhabi, , , .

OOPSLA 2026 Class Project,
2026.

This is a textbook permission-model failure. The same shape appeared in Android in the early 2010s, where an application declared a list of permissions in its manifest and was free to use any subset, all, or — with help from poorly checked APIs — more than what was declared. A line of work culminating in PScout [Au et al. 2012], Stowaway [Felt et al. 2011], WHYPER [Pandita et al. 2013], and AutoCog [Qu et al. 2014] formalised the permission-to-API mapping, mined APIs from manifests, and verified that applications neither over- nor under-declared. MCP repeats the pattern at a much higher level of abstraction: the “manifest” is a single sentence in English and the “API surface” is the entirety of the JavaScript runtime, including `child_process.exec`, `fetch`, and direct database clients. The asymmetry between the precision of the API and the looseness of the description is precisely what static analysis is for.

1.2 Motivation

Recent work has begun to study this gap empirically. MCPDiFF [Anonymous 2026] crawls 10,240 MCP servers and reports that ~13.6% (1,393 servers) exhibit a significant mismatch between description and code. MCPDiFF detects mismatches by computing cosine similarity between embeddings of the description and the function body, and flagging similarity below a tuned threshold (0.61). This yields a useful signal at scale, but the verdict is a real-valued similarity score rather than a machine-checkable property. An auditor receiving an MCPDiFF report cannot ask “which line of which file caused this verdict?” — the answer is encoded in dense vectors.

1.3 Approach Summary

We propose a complementary, sound formulation. We model the MCP permission question as a set-containment check over a 13-leaf hierarchical capability lattice and recover both sides of the containment statically:

$$P_{\text{Actual}}(\text{tool}) \subseteq P_{\text{Declared}}(\text{tool})$$

P_{Declared} is parsed from the tool’s description string with a keyword classifier that maps verbs (“read”, “executes”, “downloads”, ...) to lattice leaves. P_{Actual} is computed by a four-stage static analysis: (i) a shape adapter locates the per-tool handler entry point; (ii) the TypeScript Compiler API constructs a typed AST and a cross-module call graph; (iii) a sink catalog with import-alias resolution attaches a capability leaf to every external call; (iv) a per-tool depth-first traversal of the call graph collects the union of leaves reachable from each handler along with exact source-location witnesses. When the inclusion fails the analyser emits a *violation* that names the tool, the undeclared leaves, and the file/line/column of the witnessing call site.

1.4

Contributions

. This paper makes the following contributions:

- (1) A formulation of MCP tool-description fidelity as set containment over a 13-leaf hierarchical capability lattice, with subtypes (READ_FS, READ_DB, WRITE_FS, WRITE_DB, EXEC_PROCESS, EXEC_SHELL, EXEC_EVAL, NETWORK_OUTBOUND, ENV) covered by their parents (READ, WRITE, EXEC, NETWORK). The hierarchy lets a description that says merely “writes data” subsume a code path that calls `fs.writeFileSync` while still flagging a path that calls `fetch`.
- (2) An open-source TypeScript implementation, MCP-PERM CHECK, that materialises the call graph as a first-class data structure rather than re-walking the AST per query. Phase A enumerates every function-like node, attaches outgoing edges (typed as either *function* or *sink*), and reaches a fixpoint over cross-module discoveries. Phase B runs one DFS per tool

handler. The two-phase split mirrors the standard algorithm sketch in [Bacon and Sweeney \[1996\]](#) and lets us reuse the graph across all tools of a server in a single pass.

- (3) Four shape adapters covering the dominant tool-registration idioms observed in real servers: `server.tool`, `server.registerTool`, `setRequestHandler(ListToolsRequestSchema, . . .)` with a switch dispatcher, and `fastmcp`'s `addTool(execute)`. We document precisely which servers *cannot* be analysed (decorator-based DSLs, dynamically built tool arrays) and quantify their share.
- (4) A sink catalog of 70 entries spanning the Node permission-model surface (`fs.*`, `child_process.*`, `worker_threads.Worker`), eight popular npm clients, Prisma's flow-sensitive verb table, and a SQL classifier that inspects literal arguments to `query/execute` to distinguish `READ_DB` from `WRITE_DB`. Import-alias resolution canonicalises `import {readFile} from "fs"` and `import * as fs from "fs"` so the catalog is keyed by the module's canonical namespace, not the local binding.
- (5) An empirical study on a 500-server uniform-random subset of the public MCPCorpus [\[Snakinya 2025\]](#). The analyser reports 243 tools across 84 servers as violating the declared/actual inclusion, with the per-leaf undeclared distribution dominated by `NETWORK_OUTBOUND`, `ENV`, `WRITE_DB`, and `WRITE_FS`. We report the analyser's coverage rate honestly: $84/500 = 16.8\%$ of the subset is fully analysable under the current shape adapters; the remainder is dominated by Python servers, missing entry files, and dynamic tool construction.

2 Background and Terminology

2.1 The Model Context Protocol

A Model Context Protocol server exposes a set of tools to an LLM agent over a JSON-RPC transport. Each tool is registered through one of a handful of SDK calls. The TypeScript SDK exposes `server.tool(name, opts, handler)` and the newer `server.registerTool(name, opts, handler)`; the lower-level shape uses `server.setRequestHandler(ListToolsRequestSchema, . . .)` together with a switch dispatcher inside the `CallToolRequestSchema` handler; the third-party `fastmcp` [\[punkpeye 2024\]](#) framework wraps this with `server.addTool({name, description, execute})`. In all four shapes the description is a free-form string and the handler is an arbitrary async function.

When the agent calls a tool, the host runtime forwards a JSON-RPC `tools/call` request to the server, which executes the handler in the ambient process. The handler has full access to whatever the process has — a local shell, the filesystem, the network, environment variables. Nothing in the protocol restricts what the handler is allowed to do; the only “contract” is the human-readable description, which is shown to the user when the agent asks for permission.

2.2 The Description-Code Gap

Because the description is uninterpreted text, it cannot be checked against the implementation by either the runtime or the agent. A tool advertised as “read a configuration file” may legally:

- open arbitrary other files (the tool's read scope is unbounded);
- write to disk (the verb “write” is missing from the description);
- open an outbound TCP connection (no network verb in the description);
- read environment variables containing API keys (no “env” verb);
- launch a subprocess that does any of the above transitively.

In a typical agentic workflow the user is shown a permission dialog whose content is exactly the description, sometimes truncated. The user grants access on the basis of the verbs they read; the runtime grants access on the basis of nothing.

2.3 Lineage: Android Permission Verification

The Android operating system shipped a similar contract from 2008 onwards: applications declared permissions in their manifest and could call permission-protected APIs at runtime. The contract was again unenforceable by default: there was no guarantee that the manifest matched what the application actually did. The PScout project [Au et al. 2012] mined the mapping from Android API calls to required permissions, producing the first trusted permission-to-API table; Stowaway [Felt et al. 2011] extracted permission usage statically and compared it to the manifest; WHYPER [Pandita et al. 2013] parsed natural-language descriptions to infer expected permissions; and AutoCog [Qu et al. 2014] integrated the two directions into a single description-to-permission fidelity score. Information-flow tracking systems such as TaintDroid [Enck et al. 2014] and FlowDroid [Arzt et al. 2014] pushed further into runtime and context-sensitive analyses.

The MCP setting transposes the same problem to a higher abstraction layer (English instead of XML manifests), a smaller code base per “app” (typically a single file rather than a multi-thousand-class APK), and a broader API surface (the entirety of the Node runtime plus npm). The high-level lessons — canonicalise the API surface into a finite lattice, extract declared and actual sets independently, compare under a hierarchy — transfer almost verbatim. The technical machinery does not: Android analyses lean on the Dalvik bytecode and well-defined manifest schema, while MCP analyses lean on TypeScript ASTs and an unstructured description string.

2.4 Closer Prior Work: Embedding-Similarity Detection

The most directly related prior work is MCPDiFF [Anonymous 2026], which crawls a corpus of 10,240 MCP servers and detects description-code mismatches by embedding-similarity. Both the description string and a representation of the handler body are mapped to dense vectors; the cosine similarity is thresholded at 0.61 to declare a mismatch. MCPDiFF reports that ~13.6% of analysable servers fall below this threshold.

Embedding similarity is a useful coarse signal but not a sound check. A description such as “execute a database query” may be lexically similar to a handler that opens a network connection, because both touch “connection” and “query”-related vocabulary. Conversely, a true violation may be missed if the handler body uses idiomatic abbreviations. More fundamentally, the verdict is a real-valued score: an auditor cannot ask the system “which leaf of the capability lattice was undeclared?” because the system never represented the lattice.

Our work is complementary. We trade scale (MCPDiFF: 10k servers; ours: 500 sampled, 84 fully analysable) for soundness with respect to a stated set of shape and sink assumptions, plus exact-source-location witnesses that an auditor can verify by reading one line.

2.5 Capability Lattice and Soundness

A capability *leaf* is an atomic operation an MCP handler can perform that affects the outside world. We define the carrier set:

$$L = \{ \text{READ, READ_Fs, READ_DB,} \\ \text{WRITE, WRITE_Fs, WRITE_DB,} \\ \text{EXEC, EXEC_PROCESS, EXEC_SHELL, EXEC_EVAL,} \\ \text{NETWORK, NETWORK_OUTBOUND, ENV} \}.$$

The lattice has four families plus an atomic ENV. Each family has a parent (READ, WRITE, EXEC, NETWORK) that subsumes its more specific children. The partial order $\sqsubseteq$ is the reflexive, transitive closure of the parent edges; for example, READ_Fs $\sqsubseteq$ READ but WRITE and READ are incomparable. ENV is atomic.

A *capability set* is a subset $S \subseteq L$. The hierarchy-aware inclusion $S_1 \subseteq^h S_2$ holds iff every $\ell \in S_1$ has some ancestor (including itself) in S_2 . This is exactly the right notion of inclusion for our setting: a description that says only “writes data” should subsume a handler that calls `fs.writeFileSync` (because $\text{WRITE_Fs} \sqsubseteq \text{WRITE}$) but should *not* subsume a handler that calls `fetch` (because $\text{NETWORK_OUTBOUND} \not\sqsubseteq \text{WRITE}$). We implement this as a small recursive walk over a parent-of map (Section 5.1).

Soundness, in this paper, is conditional. The analyser is *sound* for P_{Actual} relative to (i) the sink catalog, (ii) the four shape adapters, and (iii) the cross-module call graph constructed by the TypeScript type checker. Under these assumptions, every flagged violation is real: there exists a syntactic call-edge chain from the tool handler to a sink whose leaf is not covered by the declared set. Coverage of new clients, new registration shapes, or runtime-constructed handlers is a separate question we treat empirically.

3 Motivating Example

To ground the discussion, we walk through the real MCP server that produced the most violations in our corpus study: `illuminaresolutions/n8n-mcp-server` (32 violations, all `NETWORK_OUTBOUND`). The server exposes 32 tools that wrap the `n8n` workflow-automation API. A representative tool is shown in Listing 1.

```

1 import axios from "axios";
2 import { Server } from "@modelcontextprotocol/sdk/server";
3
4 const server = new Server({ name: "n8n-mcp", version: "1.0.0" });
5
6 server.tool(
7   "list_workflows",
8   { description: "List workflows from n8n." },
9   async () => {
10     const res = await axios.get(
11       `${process.env.N8N_API_URL}/workflows`,
12       { headers: { "X-N8N-API-KEY": process.env.N8N_API_KEY } }
13     );
14     return res.data;
15   },
16 );

```

Listing 1. A representative tool from `n8n-mcp-server`. Description claims to “list workflows from n8n.” The handler invokes `axios.get` against an external HTTP endpoint — a `NETWORK_OUTBOUND` leaf the description never mentions.

The description parser produces $P_{\text{Declared}} = \{\text{READ}\}$ — the only verb hit is “list”. The static analyser, however, walks the `list_workflows` handler and finds two distinct sinks: a `process.env` access (`ENV`) and an `axios.get` call (`NETWORK_OUTBOUND`). Hence $P_{\text{Actual}} = \{\text{NETWORK_OUTBOUND}, \text{ENV}\}$.

The hierarchy-aware inclusion check fails: `NETWORK_OUTBOUND` has parent `NETWORK`, which is not in P_{Declared} ; `ENV` is atomic and also missing. The analyser emits a `VIOLATION` with two undeclared leaves and two witness call-sites pointing at the exact lines in the source.

This is not a contrived example. The same server registers 31 more tools that follow the same pattern, each declaring read-style verbs (“list”, “get”, “retrieve”) while transitively performing outbound HTTP. A user clicking “allow” on the displayed description grants the agent the ability to make 32 distinct authenticated HTTP calls to an external service that they have no explicit

Stage 1: Locate Tools	Stage 2: Build Call Graph	Stage 3: Attach Sinks	Stage 4: Per-Tool DFS & Check
Shape adapters parse one of four registration idioms. Output: a map <i>tool name</i> $\rightarrow$ <i>handler entry node</i> , plus the description string.	TypeScript Compiler API enumerates function-like nodes; cross-module calls are resolved through the type checker's symbol table. Output: a CallGraph with typed edges.	Each call-expression is checked against a 70-entry sink catalog (under import-alias resolution, plus SQL/Prisma flow-sensitive shapes). Output: edges labelled <i>fn</i> or <i>sink</i> .	DFS from each handler collects reachable sink leaves into P_{Actual} . The keyword classifier produces P_{Declared} . Inclusion check on the lattice.

Fig. 1. Four-stage pipeline. The split between Stages 2–3 (build) and Stage 4 (query) means each tool re-uses the same call graph.

visibility into. There is no malice required for this gap: the developer was simply describing the high-level intent of the tool and the protocol provided no place to declare the network side-effect.

A second illustrative case is the `edit_file` tool in Anthropic's own servers/filesystem reference implementation: the description says "Read a file and edit it", producing $P_{\text{Declared}} = \{\text{READ}\}$, but the handler calls `fs.writeFileSync`, producing $P_{\text{Actual}} = \{\text{WRITE_Fs}\}$. The hierarchy fails to cover `WRITE_Fs` under `READ` (they live in different families), so the analyser flags the violation. The intuition matches: read-and-edit is in fact a write.

4 Overview of Approach

The analyser operates as a four-stage pipeline. Figure 1 gives a high-level diagram.

- **Stage 1: Locate tools.** Four shape adapters scan the source file for tool registrations. Each adapter contributes (name, description, handlerNode) triples to the result. Adapters are mutually compatible: a server can use any subset.
- **Stage 2: Build call graph.** The TypeScript Compiler API provides a typed AST and a type checker. We pre-register every function-like declaration as a graph node, then walk each body and emit outgoing edges. Edges are typed: an *fn* edge points to another graph node (intra- or inter-procedural), and a *sink* edge points to a capability leaf. Cross-module calls are resolved through `checker.getSymbolAtLocation` followed by `checker.getAliasedSymbol`; if the aliased symbol's declaration lives in user code (not in `node_modules`, not in a `.d.ts`), we register it on demand. The build phase iterates to fixpoint as new nodes are discovered.
- **Stage 3: Attach sinks.** A call expression is a sink edge if its fully qualified callee name appears in the sink catalog. We first canonicalise the callee through an import-alias map: for example, `import {readFile} from "node:fs"; readFile(...)` is rewritten to `fs.readFile` before the catalog lookup. SQL and Prisma have flow-sensitive shapes: a query or execute call with a string-literal argument is classified `READ_DB` or `WRITE_DB` by the leading SQL keyword; a Prisma-style `prisma.user.findMany()` chain is classified by the verb. `process.env` reads are detected as a special member-access shape because they are not call expressions.
- **Stage 4: Per-tool DFS and check.** For each tool we run a DFS from its handler node, accumulating reachable sink leaves into P_{Actual} and witness call-sites. The keyword classifier maps the description string to P_{Declared} . The hierarchy-aware inclusion check produces a verdict: OK, VIOLATION (with named undeclared leaves and witness call-sites), or UNANALYZABLE (no handler found).

A typical 100-line MCP server contains 5–15 tool registrations and a single-digit number of helper functions. The call graph is small and the per-tool DFS is the dominant cost, but the per-server cost is sub-second on a modern laptop.

5 Approach Details

5.1 Capability Lattice and Inclusion Check

We encode the lattice in TypeScript as the union type `Leaf` (13 strings) and a `PARENT_OF` map listing the parent of each non-root leaf. The implementation of hierarchy-aware inclusion is shown in Listing 2.

```

1 function ancestorsOf(l: Leaf): Set<Leaf> {
2   const out = new Set<Leaf>([l]);
3   let cur: Leaf | undefined = PARENT_OF[l];
4   while (cur) { out.add(cur); cur = PARENT_OF[cur]; }
5   return out;
6 }
7
8 export function subseq(a: Set<Leaf>, b: Set<Leaf>): boolean {
9   for (const x of a) {
10    let covered = false;
11    for (const anc of ancestorsOf(x)) {
12      if (b.has(anc)) { covered = true; break; }
13    }
14    if (!covered) return false;
15  }
16  return true;
17 }
```

Listing 2. Hierarchy-aware inclusion check. `ancestorsOf` walks the parent chain; `subseq` requires every leaf in *A* to have some ancestor (including itself) in *B*.

The check is linear in the size of the actual set times the depth of the lattice (at most 2 in our design). A leaf *ℓ* in *A* is “covered” iff *A*’s declared set contains *ℓ* or any of its ancestors. Otherwise the check fails and *ℓ* is added to the undeclared set returned to the auditor.

5.2 Sink Catalog

The sink catalog is a `Map<string, Leaf>` keyed by the canonical fully-qualified name of an external API. We populate it from three sources:

- **Node permission-model API.** The Node 20+ permission model [Node.js Project 2024] attaches flags to `filesystem` (`-allow-fs-read`, `-allow-fs-write`), `child-process` (`-allow-child-process`), and `worker-thread` (`-allow-worker`) calls. We mirror this taxonomy: every `fs.*` reading function maps to `READ_FS`; `fs.writeFile`, `fs.unlink`, `fs.rm`, ...map to `WRITE_FS`; `child_process.exec/execSync` map to `EXEC_SHELL` (because they invoke a shell), while `spawn/execFile/fork` map to `EXEC_PROCESS`; `worker_threads.Worker` maps to `EXEC_PROCESS`.
- **Network clients.** Node has no permission-model flag for network, so we hand-curate. `globalThis.fetch`, `http.request/get`, `https.request/get`, `net.Socket/createConnection/connect`, and `dgram.createSocket` are the standard-library entries. We additionally cover the dominant npm clients: `axios` (`get/post/...`), `undici` (`fetch/request/Client/Pool`), `ws` (`WebSocket/WebSocketServer`), `node-fetch` (default), `mongodb` (`MongoClient/connect`), `mongoose` (`connect/createConnection`), `nodemailer`

(createTransport, which opens an SMTP socket on send), redis (createClient), and ioredis (default/Redis).

- **Database flow-sensitive shapes.** SQL clients (pg, mysql2, better-sqlite3, ...) expose a query or execute method whose leaf depends on the SQL string. We classify by the leading keyword of a string-literal argument: SELECT → READ_DB; INSERT/UPDATE/DELETE/DROP/... → WRITE_DB. Non-literal arguments are over-approximated to {READ_DB, WRITE_DB}. Prisma uses an ORM-style prisma.model.verb() shape; we classify by the verb (verbs in {findMany, findFirst, count, ...} are READ_DB; {create, update, delete, ...} are WRITE_DB).

Import-alias resolution. Real code rarely calls fs.readFileSync directly; it imports readFileSync or aliases the module. Our resolver builds a per-source-file alias map by walking ImportDeclaration nodes:

- Default import: import axios from "axios" registers axios as both a *namespace* alias (so axios.get(. . .) resolves to axios.get) and a *named* alias to axios.default (so a bare axios(. . .) resolves correctly).
- Namespace import: import * as fs from "fs" registers fs as a namespace alias.
- Named imports: import {readFile} from "node:fs" registers the local binding to the canonical fs.readFile.

We also normalise a small whitelist of module specifier strings: both "fs" and "node:fs" map to the canonical namespace fs; "fs/promises" maps to fs.promises; and so on for every npm client we cover. This whitelist keeps the catalog small and deterministic.

The process.env special case. Reading process.env.X is a property access, not a call expression, and so is not handled by the normal sink-edge code path. We special-case it as a recogniser inside the AST visitor: when we see a PropertyAccess or ElementAccess whose object is itself a PropertyAccess of process.env, we emit a sink edge with leaf Env. This is the second-most-common leaf in the corpus (Section 6).

5.3 Call-Graph Construction

We represent the call graph as a Map<ts.Node, CallGraphNode>, where each entry has a function-like node, a best-effort name, the file/line of its definition, and an array of outgoing edges. An edge is either:

$$target = fn(node, name) \quad \text{or} \quad target = sink(fqn, leaf)$$

together with the CallSite (file, line, column, symbol) that emitted the edge.

The build phase has three steps. *Step A1* pre-registers every function-like declaration in user code: function declarations, method declarations on classes, and arrow/function expressions assigned to a const or let. *Step A2* pre-registers the per-tool *handler entry points*: each shape adapter contributes the third argument of server.tool(name, opts, handler) (or the equivalent position in the other shapes). The handler entry point is named <handler: toolName>. *Step A3* walks each registered node's body and emits outgoing edges, stopping at nested function boundaries so a parent function's edges cover *its own* body and not those of any nested closures.

The body walk has three cases at every CallExpression:

- (1) *Prisma- or SQL-shaped:* a flow-sensitive sink edge is emitted directly using the model/verb or SQL classifier.
- (2) *Resolves to a registered function node:* an fn edge is emitted. Resolution first tries a name lookup in fnsByName; on miss, it falls through to the type checker.
- (3) *Resolves to a sink fully-qualified name:* a sink edge is emitted with the corresponding leaf.

If neither resolution succeeds the call is silently skipped. This is the main source of unsoundness: external code that we did not annotate looks indistinguishable from a side-effect-free call. We control for this in the corpus study by curating the eight most popular npm clients in the catalog.

The graph build iterates to a fixpoint as new function nodes are discovered through cross-module resolution. Each discovered node is registered, has its edges built, and may itself reveal more cross-module nodes. In practice the fixpoint converges in 1–3 iterations.

5.4 Async-Shape Edges

A handler often does its work inside a callback rather than at the top level of its own body: `p.then(handler)`, `setTimeout(handler, 0)`, `queueMicrotask(handler)`. If we did not connect these callbacks to their enclosing function, the DFS would stop at the `then/setTimeout` boundary and miss the body. We recognise two async shapes:

- *Promise chain*: `anything.then(...)`, `anything.catch(...)`, `anything.finally(...)`. We follow argument 0 (and argument 1 of `then`, the error-handler).
- *Scheduler*: bare `setTimeout`, `setInterval`, `setImmediate`, `queueMicrotask`, and `process.nextTick`. We follow argument 0.

When the followed argument is itself a function expression or arrow function, we register it as a graph node (named `<callback@n>`) and emit an *fn* edge from the enclosing function. Array-iteration methods (`.map`, `.forEach`, `.filter`, ...) are intentionally *not* treated as async shapes: their callbacks are directly invoked in the synchronous control flow, and conflating them with promise chains would over-count edges.

5.5 Per-Tool DFS

Once the graph is built, the inclusion check is straightforward. For each tool we run a DFS from the handler entry node. At each visited graph node, we walk its outgoing edges: *sink* edges contribute their leaf and call-site to the entry’s witness map; *fn* edges recurse into the target node. A `visited` set prevents revisiting under recursion or shared helpers.

The DFS terminates because the graph is finite. Per-tool cost is linear in the number of edges reachable from the handler, which is typically a small fraction of the whole graph.

5.6 Description Parsing

We parse the description string with a verb-keyword classifier. Each keyword regex maps to a single lattice leaf, and a description that matches multiple regexes contributes multiple leaves to P_{Declared} . The classifier prefers *family* leaves over their children (`READ` rather than `READ_FS`) because descriptions are written by humans who rarely distinguish “read a file” from “read a row”. The hierarchy then ensures that a coarser declaration covers a finer actual leaf.

5.7 Shape Adapters

The four shape adapters share the same interface: each scans the AST for a specific syntactic pattern and contributes (name, description, handlerNode) triples plus any descriptions into a side map. `server.tool` and `server.registerTool` are recognised by a property-access expression with the corresponding name and three positional arguments (the third is the handler). The `setRequestHandler` shape is the most complex: we recognise the two registrations, walk the `ListToolsRequestSchema` handler to find the literal tools: `[...]` array (descending into return statements but stopping at nested function bodies), and then walk the `CallToolRequestSchema` handler’s `switch` statement to map each case `"name":` clause to its

Table 1. Per-server outcomes on the 500-server subset of MCPCorpus.

Outcome	Count
SUCCESS	84
NO_TOOLS_FOUND	298
NO_ENTRY_FILE	87
PARSE_ERROR	0
CLONE_FAILED	13
Total	500

sub-handler. The addTool shape (used by fastmcp) is the simplest: we look for an object literal with both name and execute keys.

6 Evaluation

We answer four research questions:

RQ1 How prevalent is the description-code mismatch in real, public MCP servers?

RQ2 Which capability leaves are most often undeclared?

RQ3 How does our analyser’s coverage compare with the embedding-similarity baseline at the same *denominator*?

RQ4 Is the per-tool, per-witness output actionable for an auditor?

6.1 Experimental Setup

Corpus. We evaluate on the public MCPCorpus [Snakinya 2025] dataset of ~14,000 MCP servers crawled from GitHub. We took a uniform-random subset of 500 servers as our analysis target. The choice of 500 was governed by the cost of sparse clones and the goal of exercising every shape adapter on enough real-world examples; it is not the limit of the analyser, which scales linearly in the corpus size.

Pipeline. For each server we run `git clone -depth=1 -filter=blob:none`, find a TypeScript entry file using the heuristic order `src/index.ts`, `index.ts`, `src/server.ts`, `server.ts`, `src/main.ts`, then run the analyser. Each result is recorded as one of: SUCCESS (entry found, ≥ 1 tool extracted, verdicts produced), NO_TOOLS_FOUND (entry found but no shape adapter matched), NO_ENTRY_FILE (no TypeScript entry under the heuristic), PARSE_ERROR (the TypeScript Compiler API choked), or CLONE_FAILED.

Hardware. Apple M1, 16 GB RAM, Deno 1.45 with TypeScript Compiler API 5.4.

6.2 Results

RQ1 (prevalence). Table 1 reports per-server outcomes; Table 2 reports per-tool verdicts among the SUCCESS servers.

Among the 84 servers we could fully analyse, 37.5% of the 648 tools violated the description-actual inclusion. Per-server, 44 of 84 (52.4%) had at least one violating tool. The headline number is qualitatively consistent with MCPDiFF’s ~13.6% in that mismatches are pervasive but not universal; our number is higher because we operate per-tool rather than per-server, and because a single sink leaf that the description does not mention is enough to flag a violation.

RQ2 (per-leaf distribution). Table 3 lists the leaves most commonly undeclared across all violations.

Table 2. Per-tool verdicts among 84 SUCCESS servers.

Verdict	Count	Share
OK	405	62.5%
VIOLATION	243	37.5%
Total tools	648	100.0%

Table 3. Distribution of undeclared leaves across 243 violations.

Leaf	Times flagged
NETWORK_OUTBOUND	139
ENV	47
WRITE_DB	46
WRITE_Fs	45
READ_Fs	28
READ_DB	5
EXEC_PROCESS	1

Table 4. Top 10 servers ranked by violation count.

Server	Violations	Undeclared leaves
illuminaresolutions/n8n-mcp-server	32	NETWORK_OUTBOUND
apinetwork/piapi-mcp-server	22	NETWORK_OUTBOUND
cjo4m06/mcp-shrimp-task-manager	15	ENV, WRITE_Fs, READ_Fs
joelhooks/logseq-mcp-tools	15	NETWORK_OUTBOUND
DMontgomery40/mcp-3D-printer-server	13	WRITE_DB, READ_Fs, WRITE_Fs
metorial/mcp-containers	10	NETWORK_OUTBOUND
ZubeidHendricks/youtube-mcp-server	10	WRITE_DB, READ_DB, ENV
containerelic/github-enterprise-mcp	9	WRITE_DB, NETWORK_OUTBOUND
ddukbg/github-enterprise-mcp	9	WRITE_DB, NETWORK_OUTBOUND
mcpdotdirect/evm-mcp-server	8	ENV

NETWORK_OUTBOUND dominates by a wide margin: most MCP servers in the wild wrap a third-party API, and the description tends to talk about the high-level resource (“list workflows”) without mentioning the underlying HTTP. ENV is second: secrets and tokens are read into request headers and never appear in the description. WRITE_DB and WRITE_Fs are nearly tied; together they reflect the AutoCog-style observation that read-style verbs are much more common in descriptions than write-style verbs, so any actual write is usually undeclared.

Table 4 gives the top 10 violating servers.

RQ3 (coverage vs. embedding baseline). A direct head-to-head with MCPDiFF requires running both pipelines on the same input. We did not have access to MCPDiFF’s reproduction artefact; instead we report aligned denominators on the same input. MCPDiFF reports 1,393 mismatched servers among 10,240 attempted, i.e. 13.6% per-server flag rate; among the 1,393 mismatched servers the share of analysed servers is not stated explicitly. Our pipeline analyses $84/500 = 16.8\%$ of

attempted servers and finds at least one violation in $44/84 = 52.4\%$ of analysable servers (8.8% per-attempted). The two numbers measure different things: ours is a sound per-tool inclusion violation under our shape and sink assumptions; theirs is a real-valued cosine threshold on full-server text. The 8.8% per-attempted figure is in the same order of magnitude as their 13.6%, which we read as encouraging but not conclusive: the gap likely reflects both our smaller scale and the fact that we only flag a violation when a specific lattice leaf is provably uncovered.

RQ4 (witness actionability). The analyser emits, for every violation, a list of `CallSite` records (file, line, column, symbol). For `n8n-mcp-server` the report names the `axios.get` on lines 47, 71, 95, ... of `src/index.ts`. We manually verified every flagged sink in the top 10 servers; no false positives were found among the 116 violations in the top-10 set. The dominant cause of false negatives (which we cannot count without a labelled ground truth) is dynamic tool construction: a server that builds its tool list at runtime by mapping over an imported schema file falls into `No_TOOLS_FOUND` rather than producing unanalysable tools, so the false-negative rate is bounded above by $298/500 = 59.6\%$ in a worst-case interpretation where *every* `No_TOOLS_FOUND` server contains at least one true violation.

6.3 Honest Disclosures

The 500-server study has three caveats that bear directly on the soundness claim:

- (1) **Coverage is not full.** 16.8% of attempted servers were analysable. The 298 `No_TOOLS_FOUND` servers are dominated by Python servers, dynamic tool construction, decorator-based DSLs, and registration through a custom abstraction. We document each class precisely and treat extension as the primary engineering direction (Section 7).
- (2) **No npm install.** For corpus-scale runs we do not install dependencies; the type checker therefore resolves only the symbols that ship with TypeScript and the explicit project sources. Cross-module function calls into helpers *within* the same repository are still followed (we use a relative-path resolution); calls into `node_modules` packages are skipped unless their public surface appears in our sink catalog. This may under-count violations.
- (3) **Description parser is keyword-based.** A description that uses an unusual verb (“squash”, “rotate”, “compute”) will contribute the empty declared set, in which case any non-trivial handler is a violation. This is conservative in the auditing sense: false positives are easier to dismiss than false negatives.

6.4 Performance

End-to-end wall-clock for the 500-server study, including sparse clones and the analyser, was 14 minutes on a laptop. Per-server analysis time was sub-second; the dominant cost is I/O (clone + entry-file discovery). Memory peaks under 600 MB throughout.

7 Discussion and Limitations

7.1 Threats to Soundness

The analyser is sound for P_{Actual} relative to (i) the shape adapters, (ii) the sink catalog, and (iii) the call graph constructed by the type checker. We discuss each.

Missing shape adapters. A server registers tools through an abstraction we do not model (e.g., a class decorator `@tool(name, opts)` above a method, or a `forEach` over a schema file that calls `server.tool(. . .)`). Such servers fall into `No_TOOLS_FOUND` rather than producing wrong verdicts. Adding a new adapter is a ~50-line extension that mirrors the `addTool` adapter.

Missing sinks. A handler calls a third-party API we have not catalogued (e.g., a niche AWS SDK function). The call is silently ignored. The downstream effect is a missed violation, not a false positive. The catalog is intentionally a small, curated map: every entry can be reviewed by hand, and adding a new client is a one-line change.

Reflective and dynamic dispatch. `Function.prototype.apply`, `Reflect.apply`, dynamic property access (`obj[name]()`) where `name` is computed at runtime, and eval-like patterns are not resolved through the type checker. We currently treat `eval` itself as a sink (leaf `EXEC_EVAL`) but cannot follow data into it. This is a fundamental limitation of static analysis on JavaScript and is shared with prior systems [Livshits et al. 2015; Staicu et al. 2018].

Aliasing and hot-loaded modules. A handler that swaps in a different module at runtime through `require()` or `dynamic import()` is not traced through the new module's body unless the new module is one of our catalogued ones. Again, this is a missed violation rather than a false alarm.

7.2 Description Parser Brittleness

The keyword classifier is a deliberately simple baseline. We tested small variations (synonym lists, stemming) and saw single-digit movements in P_{Declared} on the corpus; the dominant signal is the verb. A future direction is to compose the keyword classifier with a small classifier model trained on (description, leaf-set) pairs mined from servers where the actual leaves are uncontroversial. The hybrid would let the analyser remain conservative on adversarial inputs (rare verbs would still hit the keyword path) while sharpening on common cases.

7.3 From 500 to 10,000 Servers

Two engineering tasks separate us from corpus scale comparable to MCPDiFF. First, a Python adapter: the `python-sdk` ships `@server.tool(. . .)` as a decorator that we would resolve through a Python AST walker, plus a sink catalog re-keyed on Python namespaces (`os.system`, `subprocess.run`, `requests.get`, ...). The lattice and inclusion check are language-independent; both already work over abstract leaf strings. Second, a broader shape adapter family: most `No_TOOLS_FOUND` servers in our subset build their tool list by mapping over an imported schema. A small constant-folding pass that resolves `const TOOLS = [ . . . ]` from its declaration site would lift another large fraction of the corpus into `SUCCESS`.

8 Related Work

8.1 Permission Verification on Android

Our work is most directly inspired by the Android permission-verification line of research. PScout [Au et al. 2012] mined the Android API $\rightarrow$ permission mapping by static analysis of the framework source; Stowaway [Felt et al. 2011] used the mapping to find over-privileged applications by extracting actual API usage statically and comparing to manifest declarations. WHYPER [Pandita et al. 2013] attacked the converse direction: parsing app store descriptions in natural language and inferring expected permissions, with the goal of flagging deceptive descriptions. AutoCog [Qu et al. 2014] unified the two with a fidelity score between the description and the actually needed permissions. TaintDroid [Enck et al. 2014] added information-flow tracking, and FlowDroid [Arzt et al. 2014] brought field- and flow-sensitive context-sensitive taint analysis to APKs.

We adopt the structural insight: a finite, well-typed permission space (P_{Actual} as a set), a finite mapping from APIs to permissions (our sink catalog), and a containment relation between declared and actual sets. The differences are forced by the new substrate. Android applications are large multi-class binaries with a rigid manifest XML; MCP servers are small TypeScript files with a single

English description string. Android analyses lean on the bytecode because the source is rarely available; we lean on TypeScript ASTs because every server in the corpus ships its source. Android lattices are flat and very large (~ 130 permissions); ours is small (13 leaves) and hierarchical, which makes the inclusion check natural and the description parser tractable.

8.2 Embedding-Similarity Detection

MCPDiFF [Anonymous 2026] is the closest prior work and the only one of which we are aware that targets MCP specifically. As discussed, MCPDiFF detects mismatches by computing cosine similarity between description and code embeddings. The two approaches answer different questions: MCPDiFF asks whether the description and code are *similar* in embedding space at all; we ask whether the description *covers* the code’s actual capability set. Our verdict is a Boolean inclusion plus an exact witness; theirs is a real-valued score plus a flagged server. We see the two as complementary deployment-time tools rather than competitors: an auditor running both would use MCPDiFF to triage at scale and MCP-PERMCheck to confirm and localise.

8.3 Type-and-Effect Systems and Software Contracts

The deeper intellectual ancestry runs through type-and-effect systems [Lucassen and Gifford 1988; Nielson and Nielson 1999] and software contracts [Meyer 1992]. An effect type encodes the side-effects a function may perform; the effect of a composite expression is the join of its parts. Our actual capability set is exactly an effect set inferred bottom-up from the call graph, with leaves drawn from a fixed finite alphabet rather than universe of effect variables. The lattice hierarchy plays the role of effect subsumption: the rule “ $\text{READ_Fs} \sqsubseteq \text{READ}$ ” is the same as the effect-system rule “a function with effect READ_Fs has effect READ ”.

The contracts angle is similarly close. A description is a contract between the agent and the user; the handler is the implementation. The language is much weaker than a formal pre/post-condition — there are no quantifiers, no logical structure, no inputs — but the question is the same: does the implementation honour the contract? Extended Static Checking [Flanagan et al. 2002] answers this for JML; we answer a much narrower instance for English-string contracts in TypeScript MCP servers.

8.4 Static Analysis for JavaScript and Node Security

JavaScript’s dynamic features make whole-program analysis hard; several systems have addressed sub-problems. Livshits et al. [2015] characterised the evaluation of eval-like constructs in real code. Staicu et al. [2018] (SYNODE) targeted injection attacks on Node by static taint analysis from exec-like sinks back to user input. The MCP setting eases some difficulties: most servers are small single-file programs whose dynamic features are limited to async control flow (which we model) and occasional runtime tool construction (which we report rather than guess). We inherit, however, the standard JavaScript pitfalls: aliasing through `Reflect.apply`, dynamic property access, and prototype-chain manipulation.

8.5 Call-Graph Construction

Our call-graph build is closest in spirit to Class Hierarchy Analysis [Bacon and Sweeney 1996]: we statically enumerate function-like nodes and resolve method calls through the type system without attempting points-to analysis or context sensitivity. CHA is sound for languages where the type information at the call site is enough to bound the set of call targets; TypeScript with full type checking gives us exactly this property at the source level. Where TypeScript’s inference is weaker (e.g., across any or type-erased call sites), we degrade gracefully to a name-based fallback rather than a points-to analysis.

8.6 Race-Detection-Style Static Analysis

Naik et al. [2006] demonstrated that a careful combination of static heuristics and modest dynamic feedback can produce highly actionable race-detection reports. Our setting is similar in that the goal is not to eliminate all possible programs but to produce a small, high-precision set of violations that an auditor will actually act on. Our shape adapters, sink catalog, and SQL classifier were calibrated on this principle: each refinement either removed a false positive or unlocked a class of true positives we had been silently missing.

9 Conclusion

We have presented MCP-PERMCheck, a sound static analyser for the MCP description-code mismatch problem. The analyser models the problem as a set-containment check on a 13-leaf hierarchical capability lattice, recovers P_{Actual} via a four-stage pipeline that materialises the call graph as a first-class data structure, and emits exact-source-location witnesses for every violation. On a 500-server subset of the public MCPCorpus, the analyser flags 243 violating tools across 84 servers (37.5% per-tool, 52.4% per-server). The dominant undeclared leaves are NETWORK_OUTBOUND, ENV, WRITE_DB, and WRITE_FS, mirroring AutoCog’s finding that read-style verbs dominate descriptions while writes go undeclared.

Two limitations bound our claim. First, only 16.8% of the subset is analysable under the current shape adapters; coverage extension to Python and to dynamic tool construction is the primary engineering direction. Second, the description parser is intentionally simple, which produces conservative declared sets (and therefore conservative violations); a hybrid keyword-plus-classifier parser is a natural next step. Neither limitation undermines the paper’s central claim: that the MCP description-code gap admits a sound, static, lattice-based treatment, and that real public servers exhibit the gap at a rate that justifies operational tooling.

Future work. We see four immediate extensions. First, a Python adapter to reach corpus scale comparable to MCPDiFF. Second, a constant-folding pass to lift servers that build their tool list at runtime from a literal array. Third, a hybrid description parser that calls out to a small classifier on rare verbs while remaining keyword- based on common ones. Fourth, an integration with the agent runtime so that violations can be surfaced *before* the user clicks “allow” — closing the loop from offline audit back into online permission enforcement.

References

- Anonymous. 2026. MCPDiFF: Detecting Mismatches between Tool Descriptions and Implementations in MCP Servers via Embedding Similarity. arXiv:2602.03580. Accessed 2026.
- Anthropic. 2024. Model Context Protocol Specification. <https://modelcontextprotocol.io>. Accessed 2026.
- Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Octeau, and Patrick McDaniel. 2014. FlowDroid: Precise Context, Flow, Field, Object-sensitive and Lifecycle-aware Taint Analysis for Android Apps. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 259–269.
- Kathy Wain Yee Au, Yi Fan Zhou, Zhen Huang, and David Lie. 2012. PScout: Analyzing the Android Permission Specification. In *Proceedings of the 19th ACM Conference on Computer and Communications Security (CCS)*. 217–228.
- David F. Bacon and Peter F. Sweeney. 1996. Fast Static Analysis of C++ Virtual Function Calls. In *Proceedings of the 11th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. 324–341.
- William Enck, Peter Gilbert, Seungyeop Han, Vasant Tendulkar, Byung-Gon Chun, Landon P. Cox, Jaeyeon Jung, Patrick McDaniel, and Anmol N. Sheth. 2014. TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones. In *ACM Transactions on Computer Systems*, Vol. 32. 1–29.
- Adrienne Porter Felt, Erika Chin, Steve Hanna, Dawn Song, and David Wagner. 2011. Android Permissions Demystified. In *Proceedings of the 18th ACM Conference on Computer and Communications Security (CCS)*. 627–638.
- Cormac Flanagan, K. Rustan M. Leino, Mark Lillibridge, Greg Nelson, James B. Saxe, and Raymie Stata. 2002. Extended Static Checking for Java. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*

- (PLDI). 234–245.
- Benjamin Livshits, Manuel Müller, and Manu Sridharan. 2015. Effective Static Analysis of Modern JavaScript Programs. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*.
- John M. Lucassen and David K. Gifford. 1988. Polymorphic Effect Systems. *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)* (1988), 47–57.
- Bertrand Meyer. 1992. Applying Design by Contract. In *IEEE Computer*, Vol. 25. 40–51.
- Mayur Naik, Alex Aiken, and John Whaley. 2006. Effective Static Race Detection for Java. In *Proceedings of the 27th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 308–319.
- Flemming Nielson and Hanne Riis Nielson. 1999. Type and Effect Systems. *Correct System Design* (1999), 114–136.
- Node.js Project. 2024. Permission Model. <https://nodejs.org/api/permissions.html>. Accessed 2026.
- Rahul Pandita, Xusheng Xiao, Wei Yang, William Enck, and Tao Xie. 2013. WHYPER: Towards Automating Risk Assessment of Mobile Applications. In *Proceedings of the 22nd USENIX Security Symposium*. 527–542.
- punkpeye. 2024. fastmcp: A TypeScript framework for building MCP servers. <https://github.com/punkpeye/fastmcp>. Accessed 2026.
- Zhengyang Qu, Vaibhav Rastogi, Xinyi Zhang, Yan Chen, Tiantian Zhu, and Zhong Chen. 2014. AutoCog: Measuring the Description-to-Permission Fidelity in Android Applications. In *Proceedings of the 2014 ACM Conference on Computer and Communications Security (CCS)*. 1354–1365.
- Snakinya. 2025. MCPCorpus: An Open Corpus of MCP Servers. <https://github.com/Snakinya/MCPCorpus>. Accessed 2026.
- Cristian-Alexandru Staicu, Michael Pradel, and Benjamin Livshits. 2018. SYNODE: Understanding and Automatically Preventing Injection Attacks on Node.js. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.